

Edit distance $\Rightarrow$ DP on Strings.

V. imp. question.

convert string word 1 to word 2.

word 1 $\rightarrow$ word 2.

- ① insert a char.
- ② delete a char.
- ③ replace a char.

Eg: ① $s_1 = \text{horse}$ $s_2 = \text{ros}$.

$\text{LCS} = \text{os}$. (don't touch this).

move 1: $\text{horse} \rightarrow \text{ros}$ ($h \rightarrow r$).

$\text{LCS} = \text{ros}$.

move 2, 3: Delete r and e .

$\therefore s_1 = \text{ros}$ $s_2 = \text{ros}$.

Thought

① horse
 ros

first char check.

$e \neq r$.

add 1

so, delete e .

② horse
 ros

$\rightarrow s == s$. (char's match).

so it is part of LCS.

don't touch.

$f(i-1, j-1)$

③ horse
 ros

$r \neq o$.

so, ② options.

either delete r .

or replace r with o .

At each step. $f(i, j)$. If $s_1[i] \neq s_2[j]$

- ① insert the same character
- ② delete and try finding somewhere else.
- ③ replace the char and match.

these are different allowed paths at each step

To explore all paths $\Rightarrow$ Recursion.

$\hookrightarrow$ among all the options return the case with min number of moves.

Q. How to write the Recurrence?

- ① Express everything in terms of indices $\rightarrow f(i, j)$
- ② explore all paths. (3 options here if chars don't match.)
- ③ Return min (all paths)
- ④ Base cases.

$f(i, j) \Rightarrow$ min no. of operations to convert string $s1[0 \dots i]$ to string $s2[0 \dots j]$

$\rightarrow$ this is what $f(i, j)$ signifies.

$f(i, j)$ {

Base cases:

if $(j < 0)$ return $i + 1$ ($s2$ is done so delete left in $s1$)

if $(i < 0)$ return $j + 1$ ($s1$ is done insert the remaining chars)

if $(s1[i] == s2[j])$

$\hookrightarrow$ if chars match no operation is needed

return $0 + f(i-1, j-1)$

else $\rightarrow$ chars don't match.

① Insert a new char in $s1$.

$\hookrightarrow 1 + f(i, j-1)$ (i won't change)

② delete the char in $s1$.

$\hookrightarrow 1 + f(i-1, j)$ ($i-1$ as we skipped the char.

③ replace the char in $s1$ to match.

$\hookrightarrow 1 + f(i-1, j-1)$ (since now the char matches move next in both).

return $\min(①, ②, ③)$

}

space complexity = $O(N1 * N2) + O(N1 + N2)$

$\leftarrow$ Auxiliary stack space.

To convert this to tabulation.

↳ make the dp array 1 based indexing.

↳ to enforce the base cases of j and i
 $j < 0$ and $i < 0$.

① start from the base cases and go up the way.

② express for loops of the indices or states of dp.

```
for ( $i=1 \rightarrow n_1$ ) {  
    for ( $j=1 \rightarrow n_2$ ) {  
        }  
    }
```

③ use the recurrence to get the relations.

④ return $dp[n_1][n_2]$

make the dp vector 1 based indexing to deal easily with base cases of $i < 0$ and $j < 0$.
($i = -1$ and $j = -1$)

do a index shifting by 1

$i = -1 \rightarrow i = 0$ and $j = -1 \rightarrow j = 0$
(invalid) (invalid)

wildcard matching

↳ dp on strings

this question involves string matchings.

? $\Rightarrow$ matches a single character.

* $\Rightarrow$ matches any sequence of characters.
(even empty subsequences " ")

Eg: $s = "abc"$ $p = "??*"$

abc ??*
↑ ↑
i j

if $p[j] = '*'$

* $\Rightarrow$ match empty subsequence.

↳ $f(i, j-1)$

* $\Rightarrow$ can match the current char.

↳ $f(i-1, j)$

if $p[j] = '?'$

? $\Rightarrow$ matches any single character.

↳ simply move on to the next
by matching the i th and j th char.

s: a b c
p: a b ? $\Rightarrow$? matches c in this case.

if $p[j] =$ some normal character.

↳ this must match the current
char or else return false.

$p[j]$

Now writing the Recurrence.

$f(i, j)$ {

// Base case $\Rightarrow$ must return True / False.

① If $(i < 0 \text{ and } j < 0)$ return True

② If $(j < 0)$ return False. ③ If $(i < 0)$

// Explore all possible paths or cases.

If $(p[j] == '*')$ {

return $f(i, j-1) \parallel f(i-1, j)$

else If $(p[j] == '?')$

return $f(i-1, j-1)$

else $\Rightarrow p[j]$ is a normal character.

If $(s[i] == p[j])$

return $f(i-1, j-1)$

else // current char didn't match.

return false

}

Base cases

① If $(i < 0 \text{ and } j < 0)$ return True.

$\hookrightarrow$ Note: If both s and p are processed fully then p matches s.

② If $(j < 0)$ return False

$\hookrightarrow$ Note: If pattern string is processed fully but s is left then p does not match s.

